Credit goes to,
My beloved Teacher,
Prof. Dr. Momotaz Begum, PhD (Mam)
Head of the department of CSE
DUET, Gazipur, Dhaka, Bangladesh.

Lecture-04 (Probability and other estimation)

Probability Distribution: A probability distribution is a mathematical way to show how likely different possible outcomes are for a given event.

Discrete Data (Probability Mass Function):

This is used for events with countable or finite outcomes.

Example: Flipping a coin twice. It can only get specific outcomes (0, 1 or 2 heads) and it can neatly list the probability of each outcome in a simple table.

$P(X=x) = P(x)$, where, X is a discrete random variable
x is number of heads

The sum of probabilities for all possible outcomes must equal 1.

$$\sum P(x) = 1$$

Continuous data: Probability Density Function (PDF):

This is used for data that can take any value within a range (like height, time or temperature).

All probability density functions satisfy the following conditions:

1. The random variable of Y is a function of X, $y = f(x)$
2. The value of y is greater than or equal to zero for all values of x.
3. The total area under the curve of the function is equal to one.

$$\int_{-\alpha}^{\alpha} f(x)\,dx = 1$$

Instead of checking a single point, ~~you~~ calculate the probability of the variable falling within a range [interval $[a,b]$] by calculating the area under the curve (integration)

$$P(a \leq X \leq b) = \int_{a}^{b} f(x)\,dx$$

Q: Describes the styles of probability density estimation. [DUET-2018-2019]

There are 3 styles of probability density estimation:

Parametric Estimation: This involves estimating the parameters of a chosen distribution using a data sample often through Maximum Likelihood Estimation (MLE). It is a good approach if the model actually fits the data but bad if it doesn't.

Non-parametric: This makes no assumptions about the form of the density function. Instead, it uses techniques like Kernel density estimation to fit a model to the arbitrary arbitrary distribution of the data.
Because the number of parameters grows as the dataset grows, these models can quickly hard to manage and take a long time to process new data.

Semi-Parametric: This uses mixture models to try and achieve the best of both worlds.

Types of probability Density Function:

+ Normal Distribution
- Standard Normal Distribution
- Student-t Distribution
- Chi-Square Distribution
- Continuous Uniform Distribution.

Clustering Algorithm:

Clustering is the process of dividing data points into groups so that items in the same group are much more similar to each other than they are to item in other groups.

Types of Clustering: [DUET-2018-2019]

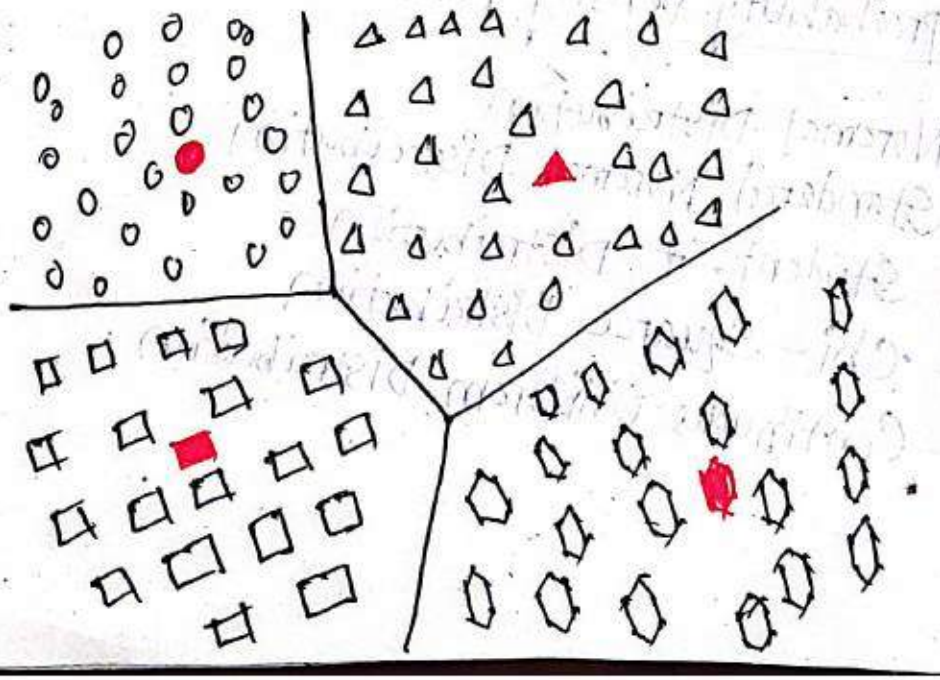
Hard Clustering: Each data point either belongs to a cluster completely or not.

Soft Clustering: Instead of forcing a point into one cluster, the algorithm assigns it a probability (likelihood) of belonging to multiple clusters.

Centroid-based Clustering:

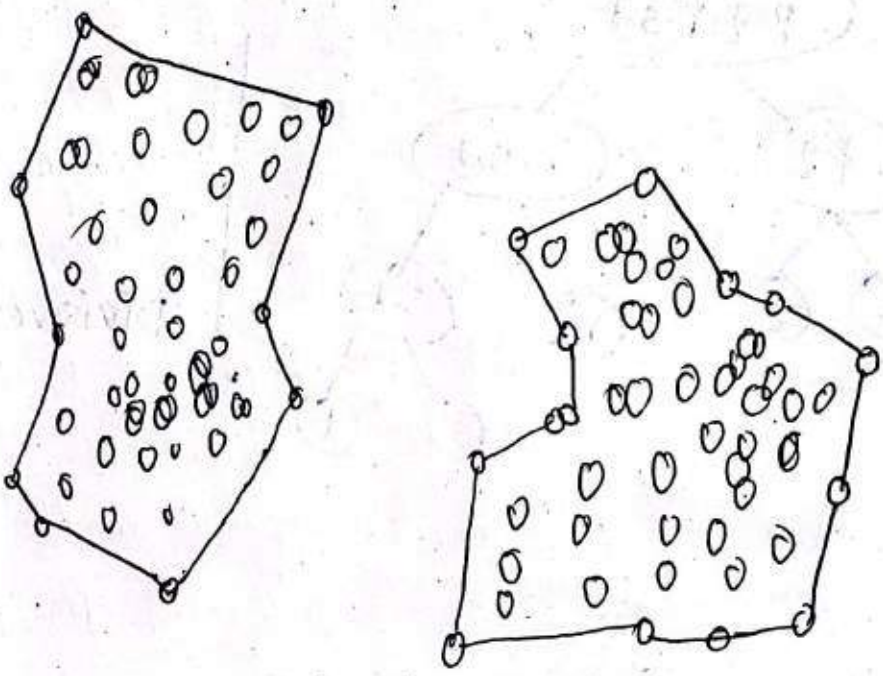
It organizes data around central points into flat, non-hierarchical groups. K-means is the most widely used example of this.

It is very efficient and simple, but it is highly sensitive to outliers and where to initial starting points.



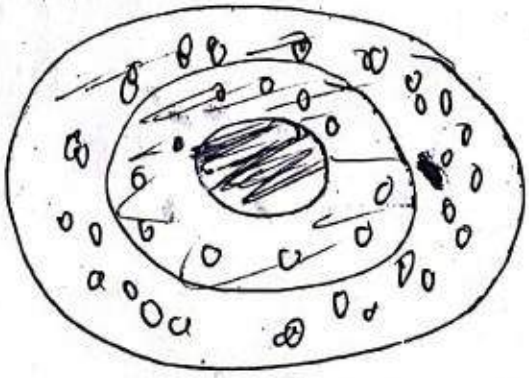
Density-Based Clustering:

It creates clusters only in regions that are ~~pac~~ packed heavily with data points.
By design, it completely ignores outliers, but it struggles heavily if data has varying levels of density or high dimensions.



Distribution-based clustering:

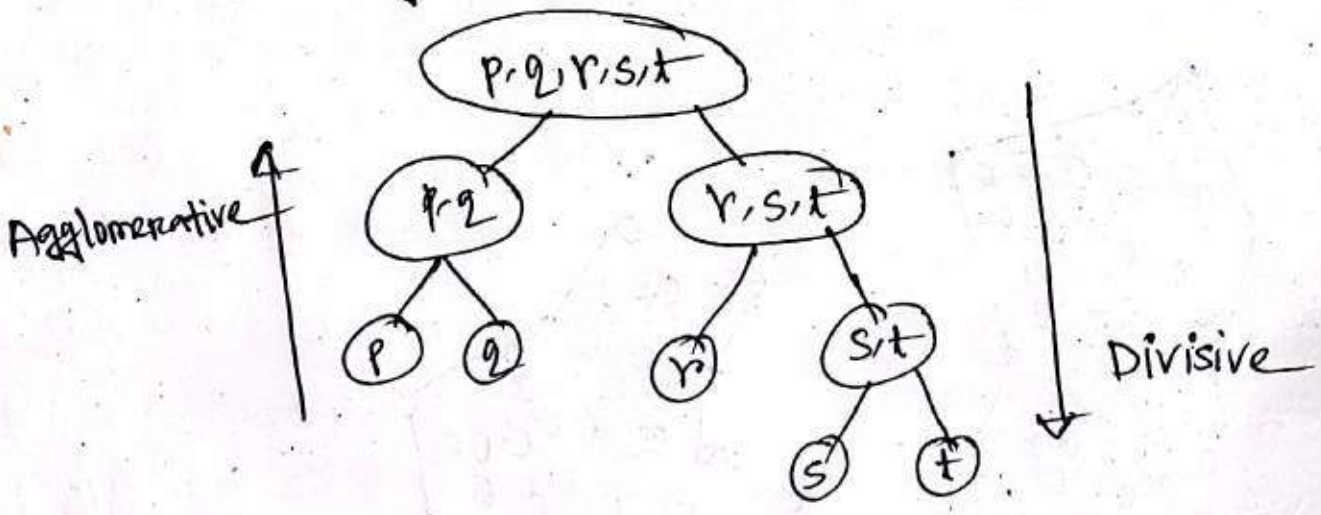
It assumes that data is made up of different probability distributions like Gaussian/Normal distributions. It groups points based on how likely they are to belong to the same distribution.



Hierarchical clustering:

It creates a tree of clusters which is perfect for data that naturally has taxonomies or levels.

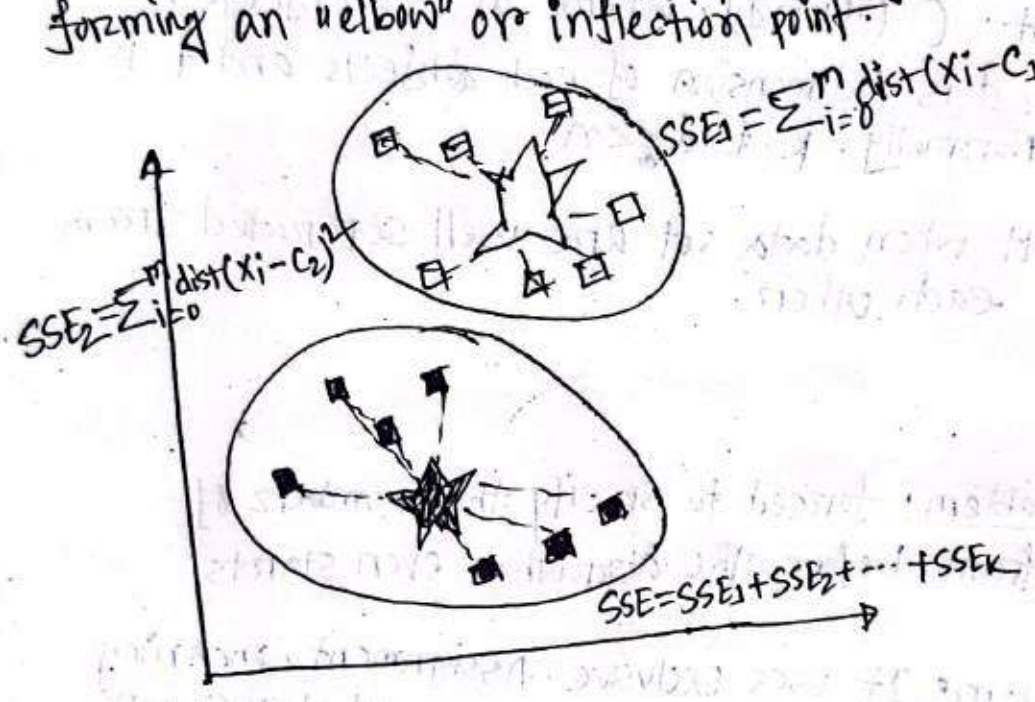
Another advantage is that any number of clusters can be chosen by cutting the tree at the right level.

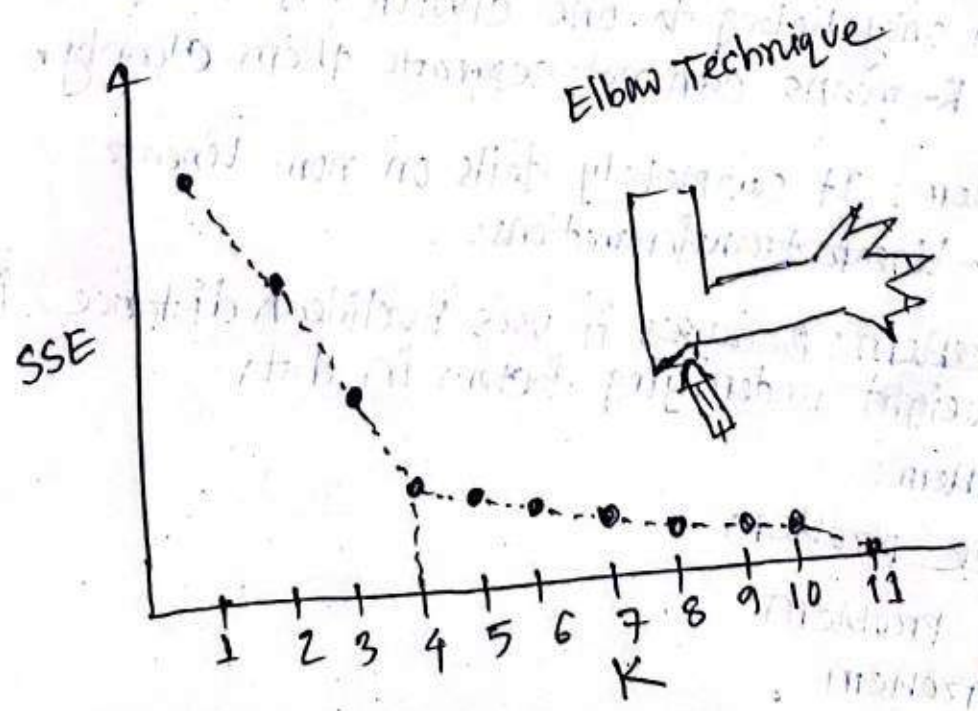


What is Elbow technique/method? [DUET-2018-2019]

Elbow method:

Used to determine the correct number of clusters (K). It calculates the sum of squared Errors (SSE) for different values of K. Plot this on a graph and choose the K-value where the line stops dropping sharply and flattens out, forming an "elbow" or inflection point.





Write the advantages and disadvantages of K-means algorithm.
[DUET-2018-2019]

Advantages

① Fast, robust and easier to understand.

② Relatively efficient: O(tknd), where n is #objects, K is #clusters, d is #dimension of each objects and t is #iterations. Normally, $K, t, d \leq n$

③ Gives best result when data set are well separated from each other.

Disadvantages

① The Guessing problem: forced to specify the number of clusters (K) upfront before the algorithm even starts.

② The overlap problem: It uses Exclusive Assignment, meaning a data point can only belong to one cluster. If two groups highly overlap, K-means can not separate them clearly.

③ The shape problem: It completely fails on non-linear datasets or non-linear transformations.

④ The distance problem: Becauses it uses Euclidean distance, it can unequally weight underlying factors in data.

⑤ The setting problem:

⑥ The random luck problem

⑦ The category problem

⑧ The "Noise" problem

Math

Cluster the following eight points (with (x,y) representing locations) into three clusters:

$A_1(2,10)$, $A_2(2,5)$, $A_3(8,4)$, $A_4(5,8)$, $A_5(7,5)$, $A_6(6,4)$, $A_7(1,2)$, $A_8(4,9)$.

Initial cluster centers are: $A_1(2,10)$, $A_4(5,8)$ and $A_7(1,2)$

The distance function between two points $a=(x_1,y_1)$ and $b=(x_2,y_2)$ is defined as: $P(a,b) = |x_2-x_1| + |y_2-y_1|$

Use K-means Algorithm to find the three cluster centers after the second iteration.

Iteration 1:

Step 1: Calculate Distance & Assign points

$A_1(2,10)$:

To $C_1(2,10)$: $|2-2| + |10-10| = 0$

To $C_2(5,8)$: $|2-5| + |10-8| = 3+2 = 5$

To $C_3(1,2)$: $|2-1| + |10-2| = 1+8 = 9$

Assign: Cluster 1

$A_2(2,5)$:

To $C_1(2,10)$: $|2-2| + |5-10| = 0+5 = 5$

To $C_2(5,8)$: $|2-5| + |5-8| = 3+3 = 6$

To $C_3(1,2)$: $|2-1| + |5-2| = 1+3 = 4$

Assign: cluster 3.

$A_3(8,4)$:

To $C_1(2,10)$: $|8-2|+|4-10| = 6+6 = 12$

To $C_2(5,8)$: $|8-5|+|4-8| = 3+4 = 7$

To $C_3(1,2)$: $|8-1|+|4-2| = 7+2 = 9$

Assign: cluster 2

$A_4(5,8)$:

To $C_1(2,10)$: $|5-2|+|8-10| = 3+2 = 5$

To $C_2(5,8)$: $|5-5|+|8-8| = 0+0 = 0$

To $C_3(1,2)$: $|5-1|+|8-2| = 4+6 = 10$

Assign: cluster 2

$A_5(7,5)$:

To $C_1(2,10)$: $|7-2|+|5-10| = 5+5 = 10$

To $C_2(5,8)$: $|7-5|+|5-8| = 2+3 = 5$

To $C_3(1,2)$: $|7-1|+|5-2| = 6+3 = 9$

Assign: cluster 2

$A_6(6,4)$:

To $C_1(2,10)$: $|6-2|+|4-10| = 4+6 = 10$

To $C_2(5,8)$: $|6-5|+|4-8| = 1+4 = 5$

To $C_3(1,2)$: $|6-1|+|4-2| = 5+2 = 7$

Assign: cluster 2

$A_7(1,2)$:

To $C_1(2,10)$: $|1-2|+|2-10| = 1+8 = 9$

To $C_2(5,8)$: $|1-5|+|2-8| = 4+6 = 10$

To $C_3(1,2)$: $|1-1|+|2-2| = 0+0 = 0$

Assign: cluster 3

A8 (4,9):

To $C_1(2,10)$: $|4-2|+|9-10| = 2+1 = 3$

To $C_2(5,8)$: $|4-5|+|9-8| = 1+1 = 2$

To $C_3(1,2)$: $|4-1|+|9-2| = 3+7 = 10$

Assign: Cluster 2

Groups after Iteration 1:

Cluster 1: {A1 (2,10)}

Cluster 2: {A3 (8,4), A4 (5,8), A5 (7,5), A6 (6,4), A8 (4,9)}

Cluster 3: {A2 (2,5), A7 (1,2)}

Step 2: Update cluster centers

New C_1: (2,10) (only one point)

New C_2: $\left(\frac{8+5+7+6+9}{5}, \frac{4+8+5+4+9}{5}\right) = \left(\frac{30}{5}, \frac{30}{5}\right) = (6,6)$

New C_3: $\left(\frac{2+1}{2}, \frac{5+2}{2}\right) = \left(\frac{3}{2}, \frac{7}{2}\right) = (1.5, 3.5)$

Iteration 2:

Step 1: Re-calculate Distance & Reassign points

A1 (2,10):

To $C_1(2,10)$: $|2-2|+|10-10| = 0+0 = 0$

To $C_2(6,6)$: $|2-6|+|10-6| = 4+4 = 8$

To $C_3(1.5,3.5)$: $|2-1.5|+|10-3.5| = 0.5+6.5 = 7$

Assign: Cluster 1

$A_2(2,5)$:

To $C_1(2,10)$: $|2-2|+|5-10| = 0+5 = 5$

To $C_2(6,6)$: $|2-6|+|5-6| = 4+1 = 5$

To $C_3(1.5,3.5)$: $|2-1.5|+|5-3.5| = 0.5+1.5 = 2$

Assign: Cluster 3

$A_3(8,4)$:

To $C_1(2,10)$: $|8-2|+|4-10| = 6+6 = 12$

To $C_2(6,6)$: $|8-6|+|4-6| = 2+2 = 4$

To $C_3(1.5,3.5)$: $|8-1.5|+|4-3.5| = 6.5+0.5 = 7$

Assign: Cluster 2

$A_4(5,8)$:

To $C_1(2,10)$: $|5-2|+|8-10| = 3+2 = 5$

To $C_2(6,6)$: $|5-6|+|8-6| = 1+2 = 3$

To $C_3(1.5,3.5)$: $|5-1.5|+|8-3.5| = 3.5+4.5 = 8$

Assign: Cluster 2

$A_5(7,5)$:

To $C_1(2,10)$: $|7-2|+|5-10| = 5+5 = 10$

To $C_2(6,6)$: $|7-6|+|5-6| = 1+1 = 2$

To $C_3(1.5,3.5)$: $|7-1.5|+|5-3.5| = 5.5+1.5 = 7$

Assign: Cluster 2

A6 (6,4):

To $C_1(2,10)$: $|6-2|+|4-10| = 4+6 = 10$

To $C_2(6,6)$: $|6-6|+|4-6| = 0+2 = 2$

To $C_3(1.5,3.5)$: $|6-1.5|+|4-3.5| = 4.5+0.5 = 5$

Assign: cluster 2

A7 (1,2):

To $C_1(2,10)$: $|1-2|+|2-10| = 1+8 = 9$

To $C_2(6,6)$: $|1-6|+|2-6| = 5+4 = 9$

To $C_3(1.5,3.5)$: $|1-1.5|+|2-3.5| = 0.5+1.5 = 2$

Assign: cluster 3

A8 (4,9):

To $C_1(2,10)$: $|4-2|+|9-10| = 2+1 = 3$

To $C_2(6,6)$: $|4-6|+|9-6| = 2+3 = 5$

To $C_3(1.5,3.5)$: $|4-1.5|+|9-3.5| = 2.5+5.5 = 8$

Assign: cluster 1 (A8 has now shifted from cluster 2 to cluster 1)

Group after iteration 2:

Cluster 1: {A1(2,10), A8(4,9)}

Cluster 2: {A3(8,4), A4(5,8), A5(7,5), A6(6,4)}

Cluster 3: {A2(2,5), A7(1,2)}

Step 2: Update Final cluster centers.

Final C_1: $\left(\frac{2+9}{2}, \frac{10+9}{2}\right) = (3, 9.5)$

Final C_2: $\left(\frac{8+5+7+6}{4}, \frac{4+8+5+4}{4}\right) = \left(\frac{26}{4}, \frac{21}{4}\right) = (6.5, 5.25)$

Final C_3: $\left(\frac{2+1}{2}, \frac{5+2}{2}\right) = (1.5, 3.5)$

Final Answer:

The three clusters after the second iteration are:

$C_1(3, 9.5)$, $C_2(6.5, 5.25)$, $C_3(1.5, 3.5)$.

Alternatives Solution

Given points	Distance from center (2,10) of cluster-01	Distance from center (5,8) of cluster-02	Distance from center (1,2) of cluster-03	point belongs to cluster
A1 (2,10)	0	5	9	C_1
A2 (2,5)	5	6	4	C_3
A3 (8,4)	12	7	9	C_2
A4 (5,8)	5	0	10	C_2
A5 (7,5)	10	5	9	C_2
A6 (6,4)	10	5	7	C_2
A7 (1,2)	9	10	0	C_3
A8 (4,9)	3	2	10	C_2

Cluster-01: First cluster contains points - $A_1(2,10)$

Cluster-02: Second cluster contains points - $A_3(8,4)$, $A_4(5,8)$, $A_5(7,5)$, $A_6(6,4)$, $A_8(4,9)$

Cluster-03: Third cluster contains points - $A_2(2,5)$, $A_7(1,2)$

Now, we re-compute the new cluster center.

For cluster-01:
We have only one point $A_1(2,10)$ in cluster-01. So cluster center remains the same.

For cluster-02:
Center of cluster-02 $= \left(\frac{(8+5+7+6+4)}{5}, \frac{(4+8+5+4+9)}{5}\right)$

$= \left(\frac{30}{5}, \frac{30}{5}\right) = (6,6)$

For cluster-03:
Center of cluster-03 $= \left(\frac{(2+1)}{2}, \frac{(5+2)}{2}\right) = \left(\frac{3}{2}, \frac{7}{2}\right) = (1.5, 3.5)$

This is completion of iteration-01

[redacted]

P-T.O

Given points	Distance from center (2,10) of cluster-01	Distance from center (6,6) of cluster-02	Distance from center (1.5,3.5) of cluster-03	point belongs to cluster
A1(2,10)	0	8	7	C_1
A2(2,5)	5	5	2	C_3
A3(8,4)	12	4	7	C_2
A4(5,8)	5	3	8	C_2
A5(7,5)	10	2	7	C_2
A6(6,4)	10	2	5	C_2
A7(1,2)	9	9	2	C_3
A8(4,9)	3	5	8.	C_1

Cluster-01: First Cluster contains points – A1(2,10), A8(4,9)

Cluster-02: Second cluster contains points – A3(8,4), A4(5,8), A5(7,5), A6(6,4)

Cluster 03: Third cluster contains points – A2(2,5), A7(1,2)

Now recalculate the new center:

For cluster 01: $\left(\frac{2+4}{2}, \frac{10+9}{2}\right) = (3, 9.5)$

For cluster 02: $\left(\frac{8+5+7+6}{4}, \frac{4+8+5+4}{4}\right) = (6.5, 5.25)$

For cluster 03: $\left(\frac{2+1}{2}, \frac{5+2}{2}\right) = (1.5, 3.5)$

This is completion of iteration – 01

Suppose there are four types of medicines/objects a, b, c, d. Each object has 2 attributes/features (pH and Weight index) are as below. Which are to be 2 groups, K=2 based on their features. Now, determine this group using K-means algorithm. [DUET-2018-2019]

medicine type	Weight index	pH
a	1	1
b	2	3
c	4	5
d	3	1

For distance we use squared Euclidean Distance:

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

Data points (weight index, pH):

a(1,1), b(2,3), c(4,5), d(3,1)

Initial clusters center:

$c_1 = a(1,1)$

$c_2 = b(2,3)$

Iteration 1:

Step 1: Calculate Distances & Assign points

Point a(1,1):

To $c_1(1,1)$: $(1-1)^2 + (1-1)^2 = 0+0 = 0$

To $c_2(2,3)$: $(1-2)^2 + (1-3)^2 = 1+4 = 5$

Assign: cluster 1

Point b(2,3):

To $C_1(1,1)$: $(2-1)^2+(3-1)^2 = 1+4 = 5$

To $C_2(2,3)$: $(2-2)^2+(3-3)^2 = 0+0 = 0$

Assign: cluster 2

Point C(4,5):

To $C_1(1,1)$: $(4-1)^2+(5-1)^2 = 9+16 = 25$

To $C_2(2,3)$: $(4-2)^2+(5-3)^2 = 4+4 = 8$

Assign: cluster 2

Point d(3,1):

To $C_1(1,1)$: $(3-1)^2+(1-1)^2 = 4+0 = 4$

To $C_2(2,3)$: $(3-2)^2+(1-3)^2 = 1+4 = 5$

Assign: cluster 1

cluster after iteration 1:

cluster 1: {a(1,1), d(3,1)}

cluster 2: {b(2,3), c(4,5)}

step 2: Update cluster centers

New C_1: $\left(\frac{1+3}{2}, \frac{1+1}{2}\right) = (2,1)$

New C_2: $\left(\frac{2+4}{2}, \frac{3+5}{2}\right) = (3,4)$

Iteration 2

Step 1: Re-calculate distance & Re-assign points

Point a(1,1):

To $C_1(2,1)$: $(1-2)^2+(1-1)^2 = 1+0 = 1$

To $C_2(3,4)$: $(1-3)^2+(1-4)^2 = 4+9 = 13$

Assign: cluster 1

Point b(2,3):

To $C_1(2,1)$: $(2-2)^2+(3-1)^2 = 0+4 = 4$

To $C_2(3,4)$: $(2-3)^2+(3-4)^2 = 1+1 = 2$

Assign: cluster 2

Point c(4,5):

To $C_1(2,1)$: $(4-2)^2+(5-1)^2 = 4+16 = 20$

To $C_2(3,4)$: $(4-3)^2+(5-4)^2 = 1+1 = 2$

Assign: cluster 2

Point d(3,1):

To $C_1(2,1)$: $(3-2)^2+(1-1)^2 = 1+0 = 1$

To $C_2(3,4)$: $(3-3)^2+(1-4)^2 = 0+9 = 9$

Assign: cluster 1

After iteration 2:

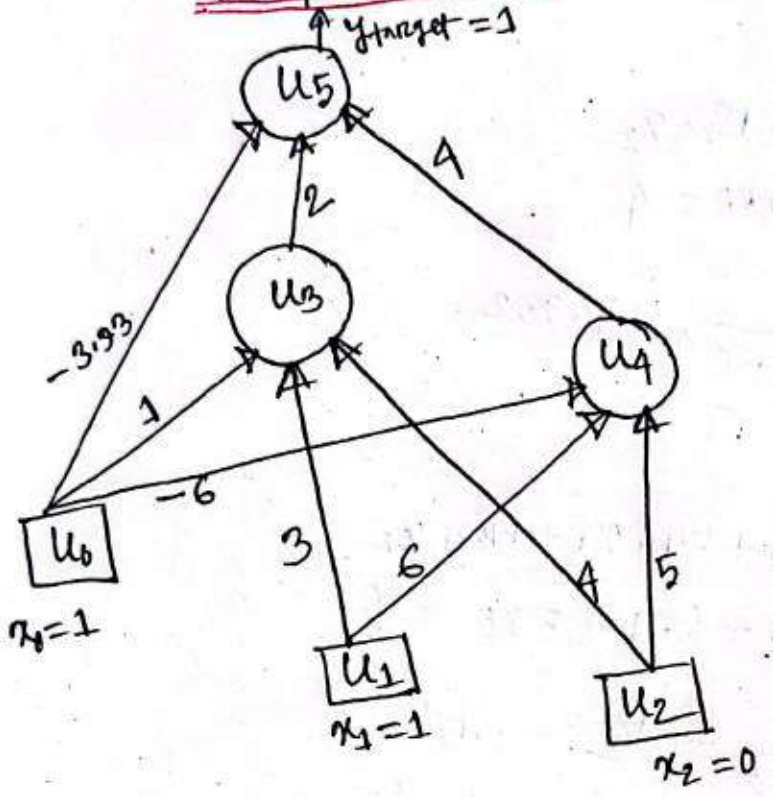
cluster 1: {a(1,1), d(3,1)}

cluster 2: {b(2,3), c(4,5)}

Since the assignments for the points did not change from iteration 1 to iteration 2, the k-means algorithm has converged and no further calculations are needed.

Lecture-05

MLP|BP Math



The network has:

Input bias: $x_0 = 1$, $x_1 = 1$, $x_2 = 0$

target output: $y_{target} = 1$

Initial weights:

For hidden unit u_3:

$W_{03} = 1$

$W_{13} = 3$

$W_{23} = 4$

For hidden unit u_4:

$W_{04} = -6$

$W_{14} = 6$

$W_{24} = 5$

For output u_5:

$W_{05} = -3.93$ (but for calculation -3.92)

$W_{35} = 2$

$W_{45} = 4$

The activation function is the sigmoid,

$$y = \frac{1}{1+e^{-a_j}}$$

where a is the weighted sum going into a neuron.

$$a_j = \sum_i W_{ij} x_i$$

Forward pass

Compute hidden unit u_3:

$$a_3 = w_{03}x_0 + w_{13}x_1 + w_{23}x_2$$
$$= 1*1 + 3*1 + 4*0 = 4$$

Now apply sigmoid:
$$y_3 = \frac{1}{1+e^{-4}} = 0.982$$

Compute hidden unit u_4:

$$a_4 = w_{04}x_0 + w_{14}x_1 + w_{24}x_2$$
$$a_4 = -6*1 + 6*1 + 5*0 = 0$$

Now, apply sigmoid: $y_4 = \frac{1}{1+e^0} = 1/2 = 0.5$

Compute output unit u_5:

$$a_5 = w_{05}x_0 + w_{35}.y_3 + w_{45}y_4$$
$$= -3.93*1 + 2*0.982 + 4*0.5$$
$$= 0.044$$

Now apply sigmoid:
$$y_5 = \frac{1}{1+e^{-0.044}} \approx 0.510$$

Compute the error:

Target is 1, actual output is 0.510

$$\text{error} = y_{\text{target}} - y_5 = 1 - 0.510 = 0.490$$

So the current output is wrong by 0.490.

Backward pass

Compute delta values starting at the output: (u_5)

$$\delta_5 = y_5(1-y_5)(y_{target}-y_5)$$
$$= 0.51(1-0.51)*(1-0.510)$$
$$= 0.1225$$

[It tell us how strongly neuron 5 should correct its incoming weights. A bigger delta means a bigger correction]

Note:
step-1
Output delta = $y(1-y)(t-y)$
[This tell us how wrong the final output is]
step-2
Then send that error backward to the hidden neurons:
Hidden delta = $y(1-y)$ (next delta × next weight)
or, $\delta_4 = y_4(1-y_4)(\delta_5 w_{45})$
This tells each hidden neuron how much it contributed to the output error.
Step3:
Update each weight using.
$W(new) = W(old) + \eta \delta X$ (Input from previous neuron)

② Hidden neuron delta (u_4): δ_4

Formula: $\delta_j = y_j(1-y_j)\sum(\delta_k w_{jk})$

Since neuron 4 connects only to output neuron 5 here, the sum becomes just one term:

$$\delta_4 = y_4(1-y_4)(\delta_5 w_{45})$$
$$= 0.5(1-0.5)*(0.1225*4)$$
$$= 0.1225$$

[Neuron 4 is hidden, so it does not compare itself directly with the target. Instead, it get its error from the output neuron. That is why we multiply by δ_5 and by the connection weight w_{45}]

③ Hidden neuron delta(u_3) δ_3

$$\delta_3 = y_3(1-y_3)(\delta_5 w_{35})$$
$$= 0.982(1-0.982)(0.1225 \times 2)$$
$$= 0.0043$$

Update Weights Using Generalized Delta Rule (BP)

Update rule:

$$\Delta W_{ij} = \eta \, \delta_j \, y_i$$

and then:

$$W_{ij}(new) = W_{ij}(old) + \Delta W_{ij}$$

Here, η = learning rate

δ_j = delta of the neuron receiving the weight

y_i = output coming from the previous neuron.

learning rate, $\eta = 0.1$

bias input, $x_0 = 1$

Update the weights into neuron H_3

These weights are:

W_{03}, W_{13}, W_{23}

Because they go into neuron 3.

all of them use:

$$\delta_3 = 0.0043$$

Memory trick

1. look at the destination neuron delta
2. multiply by ~~or~~ the source value
3. multiply by 0.1
4. add that change to the old weight

Update W_{03}

$$\Delta W_{03} = \eta \, \delta_3 x_0$$
$$= 0.1 \times 0.0043 \times 1$$
$$= 0.00043$$

$$\therefore W_{03}(new) = W_{03}(old) + \Delta W_{03}$$
$$= 1 + 0.00043$$
$$\simeq 1.0004$$

Update W_{14}

$$\Delta W_{14} = \eta \delta_4 x_1$$
$$= 0.1 \times 0.1225 \times 1$$
$$= 0.01225$$

$$\Delta W_{14}(new) = \Delta W_{14}(old) + \Delta W_{14}$$
$$= 6 + 0.01225$$
$$\approx 6.0123$$

Update W_{24}

$$\Delta W_{24} = \eta \delta_4 x_2$$
$$= 0.1 * 0.1225 * 0 = 0$$

$$W_{24}(new) = W_{24}(old) + \Delta W_{24}$$
$$= 5 + 0 = 5.000$$

Update weights (w_5) into neuron 5:

These weights are, W_{05}, W_{35}, W_{45}

Use $\delta_5 = 0.1225$

Bias input, $x_0 = 1$

$$\Delta W_{05} = \eta \delta_5 x_0$$
$$= 0.1 * 0.1225 * 1$$
$$= 0.01225$$

$$W_{05}(new) = W_{05}(old) + \Delta W_{05}$$
$$= -3.92 + 0.01225$$
$$\approx -3.9078$$

Update weight W_{35}

Source hidden output $y_3 = 0.982$

$$\Delta W_{35} = \eta \delta_5 y_3$$
$$= 0.1 * 0.1225 * 0.982$$
$$= 0.0120$$
$$W_{35}(new) = W_{35}(old) + \Delta W_{35}$$
$$= 2 + 0.0120$$
$$= 2.0120$$

Update weight W_{45}

Hidden output $y_4 = 0.5$

$$\Delta W_{45} = \eta \delta_5 y_4$$
$$= 0.1 \times 0.1225 * 0.5$$
$$= 0.006125 \approx 0.0061$$
$$W_{45}(new) = W_{45}(old) + \Delta W_{45}$$
$$= 4 + 0.0061 = 4.0061$$

Verify by doing one more forward pass

New a_3

$$a_3(new) = W_{03}(new) \cdot x_0 + W_{13}(new) \cdot x_1 + W_{23}(new) \cdot x_2$$
$$= 1.0009 * 1 + 3.0004 * 1 + 4.000 * 0$$
$$= 4.0008$$
$$y_3(new) = \frac{1}{1 + e^{-4.0008}}$$
$$= 0.9820$$

new u_4

$$a_{4(new)} = w_{04(new)} \cdot x_0 + w_{14(new)} \cdot x_1 + w_{24(new)} \cdot x_2$$

$$= -5.9878 * 1 + 6.0123 * 1 + 5.030 * 0$$

$$= 0.0245$$

Activation function:

$$y_{4(new)} = \frac{1}{1+e^{-0.0245}} \approx 0.5061$$

new u_5 (Final output)

$$a_{5(new)} = w_{05(new)} \cdot x_0 + w_{35(new)} \cdot y_{3(new)} + w_{45(new)} \cdot y_{4(new)}$$

$$= -3.9078 * 1 + 0.9820 * 2.0120 + 4.0061 * 0.5061$$

$$= 0.0955$$

The activation function:

$$y_{5(new)} = \frac{1}{1+e^{-0.0955}}$$

$$= 0.5239$$

$(y_{target} - y_5)$

Old error: $1 - 0.510 = 0.490$

New error: $1 - 5239 = 0.476$

has been redvced by 0.014

∴ This proves that one single iteration of backpropagation successfully moved the network closer to the desired goal.

Kohonen self-organizing Map (KSOM) Math

There are 4 vectors which are to be clustered into 2 clusters: J_1 and J_2. #1 (1,1,0,0); #2: (0,0,0,1); #3: (1,0,0,0); #4 (0,0,1,1)

Initial learning Rate: $\alpha(0) = 0.6$. Radius: $R = 0$ (This means when a cluster wins, only that specific cluster updates its weights, not its neighbors)

Initial Randomized Weight Matrix (W):

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{bmatrix} = \begin{array}{cc} J_1 & J_2 \\ \begin{bmatrix} 0.2 & 0.8 \\ 0.6 & 0.4 \\ 0.5 & 0.7 \\ 0.9 & 0.3 \end{bmatrix} \end{array}$$

SOM:

For the 1st vector (1,1,0,0) (x_1 x_2 x_3 x_4)

$$\|E(j)\| = \sqrt{\sum_{i=0}^{i=1} (w_{ij} - x_i)^2}$$

$$J_1 = \sqrt{(w_{11}-x_1)^2 + (w_{21}-x_2)^2 + (w_{31}-x_3)^2 + (w_{41}-x_4)^2}$$

$$= \sqrt{(0.2-1)^2 + (0.6-1)^2 + (0.5-0)^2 + (0.9-0)^2}$$

$$= 1.36$$

$$J_2 = \sqrt{(0.8-1)^2 + (0.4-1)^2 + (0.7-0)^2 + (0.3-0)^2}$$

$$= 0.98$$

J_2 has the smaller distance ($0.98 < 1.36$), so it is the "winning" neuron.

The update: We update only the weights for J_2 (column 2)

[redacted]

The weights on the winning unit are updated.

$$W_{ij}(new) = W_{ij}(old) + \alpha(X_{input} - W_{old})$$

$$W_{12}(new) = W_{12}(old) + \alpha(x_1 - W_{12}(old))$$
$$= 0.8 + 0.6(1 - 0.8) = 0.92$$

$$W_{22}(new) = 0.4 + 0.6(1 - 0.4) = 0.76$$

$$W_{32}(new) = 0.7 + 0.6(0 - 0.7) = 0.28$$

$$W_{42}(new) = 0.3 + 0.6(0 - 0.3) = 0.12$$

The weight matrix now becomes:

J_1	J_2
0.2	0.92
0.6	0.76
0.5	0.28
0.9	0.12

Not adapted

Adapted

For the 2nd vector (0,0,0,1),

$$J_1 = \sqrt{(0.2-0)^2 + (0.6-0)^2 + (0.5-0)^2 + (0.9-1)^2}$$
$$= 0.66$$

$$J_2 = \sqrt{(0.92-0)^2 + (0.76-0)^2 + (0.28-0)^2 + (0.12-1)^2}$$
$$= 2.2768$$

The winner: J_1 wins $(0.66 < 2.2768)$

Update J_1's weights:

$$W_{11}(new) = W_{11}(old) + \alpha(x_i - W_{11}(old))$$
$$= 0.2 + 0.6(0 - 0.2) = 0.08$$

$$W_{21}(new) = 0.6 + 0.6(0 - 0.6) = 0.24$$
$$W_{31}(new) = 0.5 + 0.6(0 - 0.5) = 0.20$$
$$W_{41}(new) = 0.9 + 0.6(1 - 0.9) = 0.96$$

The wight matrix now becomes:

J1	J2
0.08	0.92
0.24	0.76
0.20	0.28
0.96	0.12

↑ Adapted

For the 3rd vector (1,0,0,0)

$$J_1 = \sqrt{(0.08-1)^2 + (0.24-0)^2 + (0.2-0)^2 + (0.96-0)^2}$$
$$= 1.8656$$

$$J_2 = \sqrt{(0.92-1)^2 + (0.76-0)^2 + (0.28-0)^2 + (0.12-0)^2}$$
$$= 0.6768$$

The winner: J_2 wins $(0.6768 < 0.6768)$

So, We need to update J_2's weights (column 2)

$$W_{12}(new) = W_{12}(old) + \alpha(x_1 - W_{12}(old))$$
$$= 0.92 + 0.6(1 - 0.92) = 0.968$$

$$W_{22}(new) = 0.76 + 0.6(1 - 0.76) = 0.304$$

$W_{32}(new) = 0.28 + 0.6(0 - 0.28) = 0.112$

$W_{42}(new) = 0.12 + 0.6(0 - 0.12) = 0.048$

The weight matrix now becomes:

$$\begin{bmatrix} 0.08 & 0.968 \\ 0.24 & 0.304 \\ 0.2 & 0.112 \\ 0.96 & 0.048 \end{bmatrix}$$

↑ Adapted.

For the 4th vector (0, 0, 1, 1)

$J_1 = \sqrt{(0.08-0)^2 + (0.24-0)^2 + (0.2-1)^2 + (0.96-1)^2}$

$= 0.7056$

$J_2 = \sqrt{(0.968-0)^2 + (0.304-0)^2 + (0.112-1)^2 + (0.048-1)^2}$

$= 2.724$

The winner: J_1 wins $(0.7056 < 2.724)$

The update: We update J_1's weights (column 1)

$W_{11}(new) = W_{11}(old) + \alpha(x_1 - W_{11}(old))$

$= 0.08 + 0.6(0 - 0.08)$

$= 0.032$

$W_{21}(new) = 0.24 + 0.6(0 - 0.24)$

$= 0.096$

$$W_{31} = 0.20 + 0.6(1 - 0.2)$$
$$= 0.680$$

$$W_{41} = 0.96 + 0.6(1 - 0.96)$$
$$= 0.984$$

The updated weight matrix becomes:

$$\begin{bmatrix} 0.032 & 0.968 \\ 0.096 & 0.304 \\ 0.680 & 0.112 \\ 0.984 & 0.048 \end{bmatrix}$$

↑ Adapted

[redacted]

Recall Mode

$$J = W^T X^T$$

$$= \begin{bmatrix} 0.032 & 0.968 \\ 0.096 & 0.304 \\ 0.680 & 0.112 \\ 0.984 & 0.048 \end{bmatrix}^T \times \begin{bmatrix} 1,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,1,1 \end{bmatrix}^T$$

$$= \begin{bmatrix} 0.032 & 0.096 & 0.680 & 0.984 \\ 0.968 & 0.304 & 0.112 & 0.048 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Bidirectional Associative Memory (BAM)

It is a type of artificial neural network that acts like a two-way memory system.

Connecting Different things:

Unlike some memory systems that just fix a broken version of a single pattern, a BAM links an input pattern to a completely different output pattern.

The "Bidirectional" Bouncing:

It has two layers: an input layer and an output layer. When you give it an input, it sends that information forward to create an output. Then, it sends that output backward to create a new input and repeats this back and forth process. It keep bouncing the information back and forth between the two layers until the pattern stops changing and settles into a "stable point".

Handling Messy Data:

One of the best features of a BAM is its ability to find the correct matching pattern even if the starting input you give it is noisy, corrupted or incomplete.

Technical details:

The connections (weights) that carry the information forward are the exact mathematical flip (transpose) of the connections that carry the information backward.

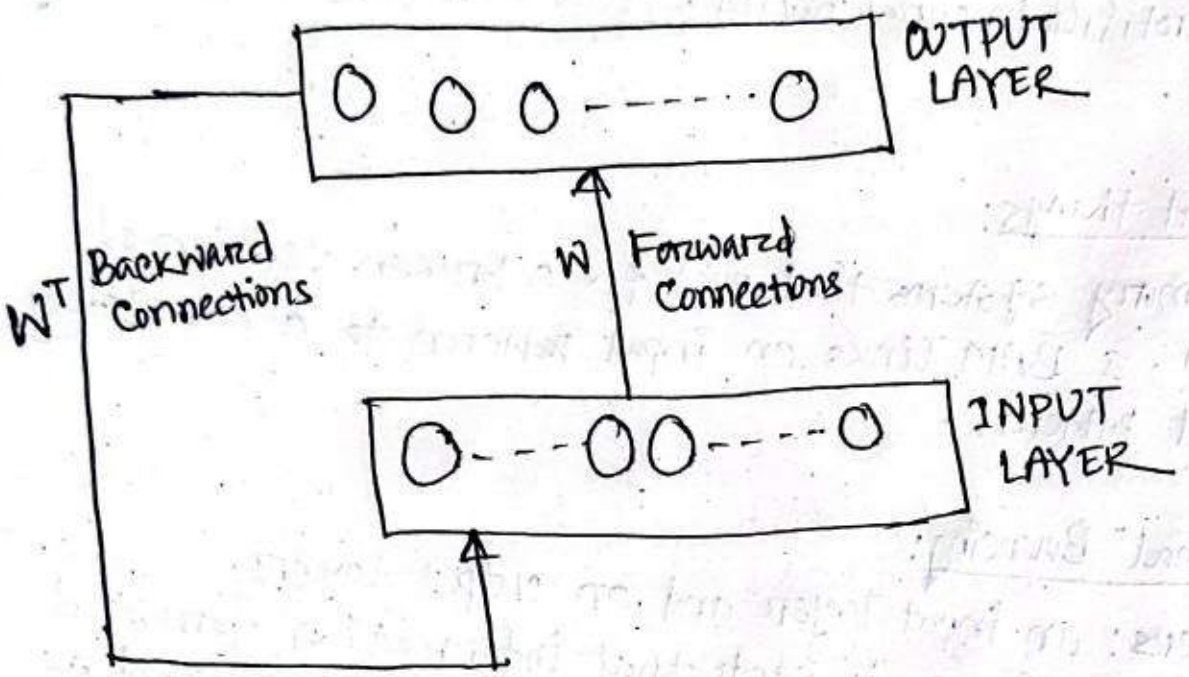


Fig: The bidirectional associative memory

BAM Algorithm

Weight Assignments:

$$W_{ji} = \sum_{p=1,q=1}^{m} P_{i,p} Q_{j,q}$$

Where, Stores m pairs of patterns.

P & q form an association pair then set a backward weight $W_{ij} = a$ forward weight W_{ji}

Calculation of Activation

Initially input units at time 0 with

$$O_i(0) = P_i$$

P_i is the ith component of the input pattern.

At time = t $(t>0)$

$$O_j(t+1) = F_h\left(\sum W_{ji} O_i(t)\right)$$

Where, $O_j(t)$ is the activation level of unit J at time t and function F_h is a hard limiting function

$$F_h(a) = \begin{cases} 1, & a>0 \\ -1(0), & a<0 \\ O_j(t) \text{ (i.e unchanged)}, & a=0 \end{cases}$$

BAM Math

Use a Bidirectional associative memory (BAM) to store three pairs of vectors as follows: [DUET-2020-2021]

Original Vector	Associated Vector
$A_1 = [1, -1, -1]$	$B_1 = [1, -1]$
$A_2 = [-1, 1, -1]$	$B_2 = [-1, 1]$
$A_3 = [-1, -1, 1]$	$B_3 = [1, -1]$

Use the Outer product to construct weight matrix. And prove that $A_1 = B_1$ and vice versa.

Soln: Use the outer products to construct the weight matrix,

$$W = \sum_i A_i^T B_i$$

N.B: Outer product means we take a column ('A' pattern) and multiply by a row ('B' pattern) create a grid/matrix

That is, $W = A_1^T B_1 + A_2^T B_2 + A_3^T B_3$

$$A_1^T B_1 = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix} \times [1, -1]$$

Row1: $1 \times 1 = 1$; $1 \times -1 = -1$

Row2: $-1 \times 1 = -1$; $-1 \times -1 = 1$

Row3: $-1 \times 1 = -1$; $-1 \times -1 = 1$

$$\therefore A_1^T B_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$A_2^T B_2 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} \times [-1, 1]$$

Row1: $-1 \times -1 = 1$; $-1 \times 1 = -1$

Row2: $1 \times -1 = -1$; $1 \times 1 = 1$

Row3: $-1 \times -1 = 1$; $-1 \times 1 = -1$

$\therefore A_2^T B_2 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$

$A_3^T B_3 = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1, -1 \end{bmatrix}$

Row1: $-1 \times 1 = -1$; $-1 \times -1 = 1$

Row2: $-1 \times 1 = -1$; $-1 \times -1 = 1$

Row3: $1 \times 1 = 1$; $1 \times -1 = -1$

$\therefore A_3^T B_3 = \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$

$\therefore W = A_1^T B_1 + A_2^T B_2 + A_3^T B_3$

$= \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$

Top-left: $1 + 1 - 1 = 1$

Top-right: $-1 - 1 + 1 = -1$

middle left: $-1 - 1 - 1 = -3$

middle right: $1 + 1 + 1 = 3$

Bottom-left: $-1 + 1 + 1 = 1$

Bottom-right: $1 - 1 - 1 = -1$

$\therefore W = \begin{bmatrix} 1 & -1 \\ -3 & 3 \\ 1 & -1 \end{bmatrix}$

Recalling a Memory (Forward pass)

Suppose we use A_1 as the probe vector.

$$F_h(A_1 W)$$

$$\therefore A_1 W = [1, -1, -1] \times \begin{bmatrix} 1 & -1 \\ -3 & 3 \\ 1 & -1 \end{bmatrix}$$

First number: multiply A_1 by the first column of W then add:

$$(1 \times 1) + (-1 \times -3) + (-1 \times 1) = 1 + 3 - 1 = 3$$

2nd number: multiply A_1 by the second column of W, then add:

$$(1 \times -1) + (-1 \times 3) + (-1 \times -1) = -1 + 3 + 1 = -3$$

$$\therefore A_1 W = [3, -3]$$

Applying the filter: $F_h([3, -3])$

$$\therefore F_h([3, -3]) = [1, -1] = B_1$$

We know,

$$F_h(a) = \begin{cases} 1, & a > 0 \\ -1(0) & a < 0 \\ 0 \text{ (unchanged)} & a = 0 \end{cases}$$

Which is perfectly matches B_1 ~~(proved)~~

$A_1 = B_1$ (Proved)

Reverse Association (Backward Pass)

However, suppose we use B_1 as the probe vector to perform reverse association.

$$F_h(B_1 W^T)$$

$$W^T = \begin{bmatrix} 1 & -3 & 1 \\ -1 & 3 & -1 \end{bmatrix}$$

P.T.O

$\therefore B_1 W^T = [1, -1] \times \begin{bmatrix} 1 & -3 & 1 \\ -1 & 3 & -1 \end{bmatrix}$

First number: $(1 \times 1) + (-1 \times -1) = 1 + 1 = 2$

2nd number: $(1 \times -3) + (-1 \times 3) = -3 - 3 = -6$

3rd number: $(1 \times 1) + (-1 \times -1) = 1 + 1 = 2$

$B_1 W^T = [2, -6, 2]$

Applying the filter: $F_h([2, -6, 2]) = [1, -1, 1]$

The output $[1, -1, 1]$ does not retrieve $A_1 [1, -1, -1]$ correctly. More iteration may be needed. In this case, however, since $B_1 = B_3$, there is always ambiguity.

[redacted]

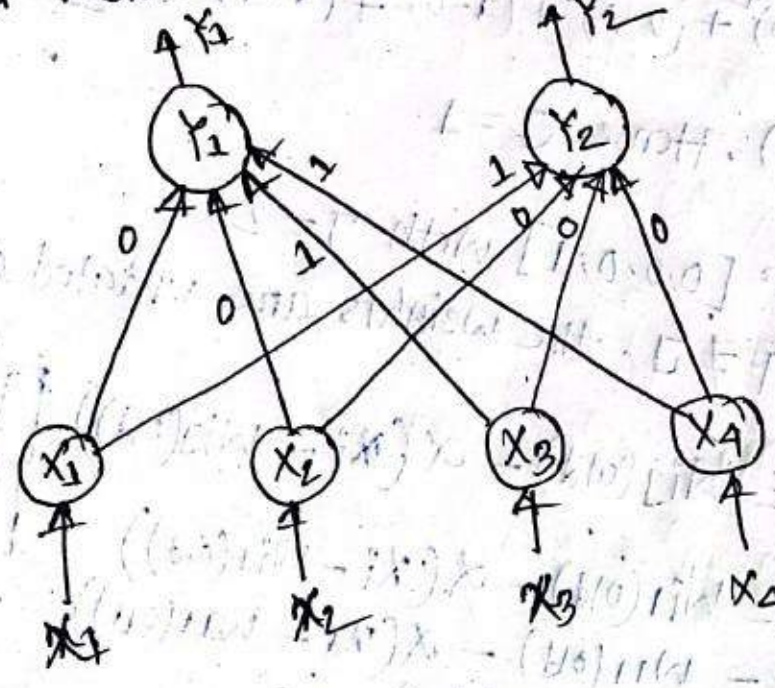
Learning Vector Quantization (LVQ) Math

Construct and test an LVQ net with five vectors assigned to two classes. The vector along with the classes are shown in table

Vector	Class
[0, 0, 1, 1]	1
[1, 0, 0, 0]	2
[0, 0, 0, 1]	2
~~[0, 1, 1, 0]~~ [1, 1, 0, 0]	1
[0, 1, 1, 0]	1

Learning rate $\alpha = 0.1$

In the given five vectors, first two vectors are used as initial weight vectors and the remaining three vectors are used as Input vectors



Soln: Initialize the reference weight vectors as:

$W_1 = [0,0,1,1]$

$W_2 = [1,0,0,0]$

Learning rate, $\alpha = 0.1$

First input vector $[0,0,0,1]$ with $T = 2$

Calculate the square of the Euclidean distance,

$$D(J) = \sum_{i=1}^{4} (W_{ij} - x_i)^2$$

$$D(J) = (W_{1J} - x_1)^2 + (W_{2J} - x_2)^2 + (W_{3J} - x_3)^2 + (W_{4J} - x_4)^2$$

$$D(1) = (W_{11} - x_1)^2 + (W_{21} - x_2)^2 + (W_{31} - x_3)^2 + (W_{41} - x_4)^2$$

$$D(1) = (0-0)^2 + (0-0)^2 + (1-0)^2 + (1-1)^2 = 1$$

$$D(2) = (W_{12} - x_1)^2 + (W_{22} - x_2)^2 + (W_{32} - x_3)^2 + (W_{42} - x_4)^2$$

$$= (1-0)^2 + (0-0)^2 + (1-0)^2 + (1-1)^2 = 2$$

$\therefore D(1) < D(2)$, Hence $J = 1$

First vector $[0,0,0,1]$ with $T = 2$

Now that $T \neq J$, the weights are updated as follows.

$$W_{ij}(new) = W_{ij}(old) - \alpha(x_i - W_{ij}(old))$$

[If the classification had been correct we would use a "+" sign instead of "−" sign]

$$W_{i1}(new) = W_{i1}(old) - \alpha(x_i - W_{i1}(old))$$

$$W_{11}(new) = W_{11}(old) - \alpha(x_1 - W_{11}(old))$$

$$= 0 - 0.1(0-0) = 0$$

$W_{21}(new) = W_{21}old - \alpha(x_2 - W_{21}(old)) = 0 - 0.1(0-0) = 0$

$W_{31}(new) = W_{31}(old) - \alpha(x_3 - W_{31}(old)) = 1 - 0.1(0-1) = 1.1$

$W_{41}(new) = W_{41}(old) - \alpha(x_4 - W_{41}(old)) = 1 - 0.1(1-1) = 1$

updated weight

$W_1 = [0, 0, 1.1, 1]$

$W_2 = [1, 0, 0, 0]$

Second Input vector $[1, 1, 0, 0]$ with $T = 1$

Calculate the square of the Euclidean distance,

$$D(J) = \sum_{i=1}^{4} (W_{ij} - x_i)^2$$

$D(J) = (W_{1J} - x_1)^2 + (W_{2J} - x_2)^2 + (W_{3J} - x_3)^2 + (W_{4J} - x_4)^2$

$D(1) = (W_{11} - x_1)^2 + (W_{21} - x_2)^2 + (W_{31} - x_3)^2 + (W_{41} - x_4)^2$

$D(1) = (0-1)^2 + (0-1)^2 + (1.1-0)^2 + (1-0)^2 = 4.21$

$D(2) = (W_{12} - x_1)^2 + (W_{22} - x_2)^2 + (W_{32} - x_3)^2 + (W_{42} - x_4)^2$

$D(2) = (1-1)^2 + (0-1)^2 + (0-0)^2 + (0-0)^2 = 1$

$D(2) < D(1)$ Hence $J = 2$

Now that $T \neq J$, the weights are updated as follows

$W_{iJ}(new) = W_{iJ}(old) - \alpha(x_i - W_{iJ}(old))$

$W_{i2}(new) = W_{i2}(old) - \alpha(x_i - W_{i2}(old))$

$$W_{12}(new) = W_{12}(old) - \alpha(x_1 - W_{12}(old))$$
$$= 1 - 0.1(1-1) = 1$$

$$W_{22}(new) = W_{22}(old) - \alpha(x_2 - W_{22}(old))$$
$$= 0 - 0.1(1-0) = -0.1$$

$$W_{32}(new) = W_{32}(old) - \alpha(x_3 - W_{32}(old))$$
$$= 0 - 0.1(0-0) = 0$$

$$W_{42}(new) = W_{42}(old) - \alpha(x_4 - W_{42}(old))$$
$$= 0 - 0.1(0-0) = 0$$

Updated weight

$W_1 = [0, 0, 1.1, 1]$

$W_2 = [1, -0.1, 0, 0]$

Third Input vector $[0, 1, 1, 0]$ with $T = 1$

Calculate the square of the Euclidean distance

$$D(j) = \sum_{i=1}^{4} (w_{ij} - x_i)^2$$

$$D(j) = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2 + (w_{3j} - x_3)^2 + (w_{4j} - x_4)^2$$

$$D(1) = (w_{11} - x_1)^2 + (w_{21} - x_2)^2 + (w_{31} - x_3)^2 + (w_{41} - x_4)^2$$
$$= (0-0)^2 + (0-1)^2 + (1.1-1)^2 + (1-0)^2 = 2.01$$

$$D(2) = (w_{12}-x_1)^2 + (w_{22}-x_2)^2 + (w_{32}-x_3)^2 + (w_{42}-x_4)^2$$

$$D(2) = (1-0)^2 + (-0.1-1)^2 + (0-1)^2 + (0-0)^2 = 3.21$$

$D(1) < D(2)$, Hence $J = 1$

Third Input vector $[0,1,1,0]$ with $T=1$

Now that $T=J$, the weights are updated as follows,

Because classification is correct that is why '+' use in the equation

$$w_{iJ}(new) = w_{iJ}(old) + \alpha(x_i - w_{iJ}(old))$$

$$w_{i1}(new) = w_{i1}(old) + \alpha(x_i - w_{i1}(old))$$

$$w_{11}(new) = w_{11}(old) + \alpha(x_1 - w_{11}(old))$$
$$= 0 + 0.1(0-0) = 0$$

$$w_{21}(new) = w_{21}(old) + \alpha(x_2 - w_{21}(old))$$
$$= 0 + 0.1(1-0) = 0.1$$

$$w_{31}(new) = w_{31}(old) + \alpha(x_3 - w_{31}(old))$$
$$= 1.1 + 0.1(1-1.1) = 1.09$$

$$w_{41}(new) = w_{41}(old) + \alpha(x_4 - w_{41}(old))$$
$$= 1 + 0.1(0-1) = 0.9$$

Updated weight

$$W_1 = [0, 0.1, 1.09, 0.9]$$

$$W_2 = [1, -0.1, 0, 0]$$

[redacted]

- The first epoch of the training has been completed
- Correct class is not obtained for first and second input patterns.
- Repeat epoch until correct class is obtained for all input patterns.

But Reduce the learning rate is $\alpha(t+1) = 0.5\,\alpha(t)$

Step 5: Now reduce the learning rate

$$\alpha(t+1) = 0.5\,\alpha(t)$$
$$\alpha(0+1) = 0.5\,\alpha(0)$$
$$\text{New}, \alpha = 0.5 \times 0.1$$
$$= 0.05$$

Step 6: Test stopping condition: This may be a fixed number of iterations (excluding step 0 of course) or learning rate reaching a sufficiently small value.

[redacted]

Write the fundamental working principles of our information processing system/ Biological Neural Network (BNN). [DUET-2018-2019]

(1) The human brain is a complicated, creative information-processing system. As technology advanced from primitive to modern, the metaphors used to describe the brain also advanced.

(2) Information processing starts with input from the sensory organs which transform physical stimuli such as touch, heat, sound waves or photons of light into electrochemical signals.

(3) The sensory information is repeatedly transformed by the algorithms of the brain in both bottom up and top-down processing. For example, when you look at a black square on a white background, bottom up processing spots the color changes and lines to figure out you are looking at a box shape. Top-down processing instantly uses those clues to decide you are looking at a solid "black box" rather than just a box-shaped hole cut out of a white wall.

(4) Once information is processed to a degree, an attention filter decides how important the signal is and which cognitive processes it should be made available to. For example, your brain has a built-in attention filter that acts like a gatekeeper for your mind. It constantly blocks out useless background noise like individual blades of grass, so you are not overwhelmed but instantly shifts your focus to highly important details like hearing your name in a noisy crowd.

⑤ In order for the brain to process information, it must first be stored. To use new information, your brain first "encodes" it (saving it by how it looks, sounds or means) into your memory. To keep that information alive, it uses looping electrical signals for quick, short-term thoughts and physically changes protein structures to lock in permanent, long-term memories.

⑥ Information processing in the brain is the topic of a large, ongoing body of research.

Adaptive Resonance Theory (ART) Math

Let us consider an ART-1 net with 5 input units and 3 cluster unit. After some training the net attains the bottom-up and top-down weights matrices as shown below.

$$B_{5\times3} = \begin{bmatrix} 0.2 & 0 & 0.2 \\ 0.5 & 0.8 & 0.2 \\ 0.5 & 0.5 & 0.2 \\ 0.5 & 0.8 & 0.2 \\ 1 & 0 & 0.2 \end{bmatrix} \quad \text{and} \quad T_{3\times5} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Show the behaviour of the net if it is presented with the training pattern $S = [0,1,1,1,1]$. Assume $L=2$ and $P=0.8$

where, L = Learning parameter
P = Vigilance parameter

[DVET-2018-2019]

Soln:

We start the training process from step 2 of ART-1 learning process. as step is just for perform all operation from step 2 to end:

Step 2: For each training pattern S, DO step 3 to 13.

Here, $S = [0,1,1,1,1]$

Step 3: Apply input pattern S to F1(a) layer. i.e set the activations of the F1(a) units to the input traing pattern S:

set the activations of the F1(a) units to $[0,1,1,1,1]$

Step 4: Set the activations of F2 layer to all 0.

$$y_1 = y_2 = y_3 = 0$$

Step 5: Find the norm of S.

$$\|s\| = \sum_{i=1}^{m} s_i = (0+1+1+1+1) = 4$$

Step 6: propagate input from $F_1(a)$ to interface layer $F_1(b)$ So that $x_i = s_i$ for all $i=1$ to m.

$$x = s = [0,1,1,1,1]$$

Step 7: Compute the net inputs to each unbibitted unit of the F_2 layer. [competing for a winner]

For $j=1$ to n DO

If $y_j \neq -1$ then $y_j = \sum_{i=1}^{m} b_{ij} \times x_i$

$$Y = [y_1, y_2, y_3] = x \times B = [0,1,1,1,1] \times \begin{bmatrix} 0.2 & 0 & 0.2 \\ 0.5 & 0.8 & 0.2 \\ 0.5 & 0.5 & 0.2 \\ 0.5 & 0.8 & 0.2 \\ 0.1 & 0 & 0.2 \end{bmatrix}$$

$y_1 : (0 \times 0.2) + (1 \times 0.5) + (1 \times 0.5) + (1 \times 0.5) + (1 \times 0.1) = 1.6$

$y_2 : (0 \times 0) + (1 \times 0.8) + (1 \times 0.5) + (1 \times 0.8) + (1 \times 0) = 2.1$

$y_3 : (0 \times 0.2) + (1 \times 0.2) + (1 \times 0.2) + (1 \times 0.2) + (1 \times 0.2) = 0.8$

$\therefore Y = [y_1, y_2, y_3] = [1.6, 2.1, 0.8]$

Step 8: While reset is True Do steps 9 to 12. [Declaring the winner]

Step 9: If $y_j = -1$ for all cluster units, then all of them are inhibited and the pattern can not be learnt by the net. Otherwise, find the J such that $y_J \geq y_j$ for all $j = 1$ to n. Then the Jth cluster unit is the winner. In case of a tie, take the smallest J

Now find the largest among three y_1, y_2, y_3.

None of the cluster units is inhibited and the cluster unit Y_2 has the largest activation 2.1 So winner is Y_2 and $J = 2$.

Step 10: The Top-down check (Expectation)

Now the network uses the top-down weights to ask, "Does this winner actually look like ~~this~~ the input?". We multiply the input pattern S by the 2nd row of the top-down matrix T (since Y_2 won)

update X: $x_i = s_i \cdot t_{Ji}$ for all $i = 1$ to m

$$X = S \cdot T_2 = [0, 1, 1, 1, 1] \cdot [0, 1, 1, 1, 0] = [0, 1, 1, 1, 0]$$

Step 11: Find the norm of x: $\|x\| = \sum_{i=1}^{m} x_i = (0+1+1+1+0) = 3$

Step 12: Test for Reset: If $\frac{\|x\|}{\|s\|} < p$ Then inhibit the Jth cluster unit by setting $y_J = -1$. Go To step 8. Otherwise

$\frac{\|x\|}{\|s\|} \geq p$ and proceed to step 13.

$\frac{\|x\|}{\|s\|} = \frac{3}{4} = 0.75 < 0.8 = \rho$ Hence y_2 must be inhibited.

$\therefore y_2 = -1$. Reset = True, Go to step 8.

Step8: While reset is True DO steps 9 to 12.

Step9: Y_2 is inhibited, we have $[y_1, y_2, y_3] = [1.6, -1, 0.8]$.
Therefore unit Y_1 has the largest activation 1.6 and winner is Y_1, so that $J = 1$.

Step10: Update x: $x_i = s_i \cdot t_{Ji}$ for all $i = 1$ to m.

$$x = s \cdot T_1 = [0,1,1,1,1] \cdot [1,1,1,1,1]$$
$$= [0,1,1,1,1]$$

Step11: Find the norm of x: $\|x\| = \sum_{i=1}^{m} x_i = (0+1+1+1+1) = 4$

Step12: Test for reset: If $\frac{\|x\|}{\|s\|} < \rho$ Then inhibit the Jth cluster unit by setting $y_j = -1$. Go to step 8
otherwise $\frac{\|x\|}{\|s\|} \geq \rho$ and proceed to step 13.

$\frac{\|x\|}{\|s\|} = \frac{4}{4} = 1 > 0.8 = \rho$, hence Reset = false

Go to step 13.

We Now update $T_{3\times5}$, the top-down weight matrix, we have $t_{ji} = t_{ij} = x_i$
So that $T_{1i}(new) = X = [0,1,1,1,1]$

$$\therefore T_{3\times5} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

#Write Advantage & Limitation/Disadvantage of ART

Advantage:

- It exhibits stability and is not distributed by a wide variety of inputs provided to its network.
- It can be integrated and used with various other techniques to give good result.
- It can be used for various fields such as mobile robot control, face recognition, target recognition, medical diagnosis, signature verification etc.
- It has got advantages over competitive learning (like bpnn etc).

Limitations of ART

Some ART networks are inconsistent (like the Fuzzy ART and ART1) as they depend upon the orders in which training data or upon the learning rate.